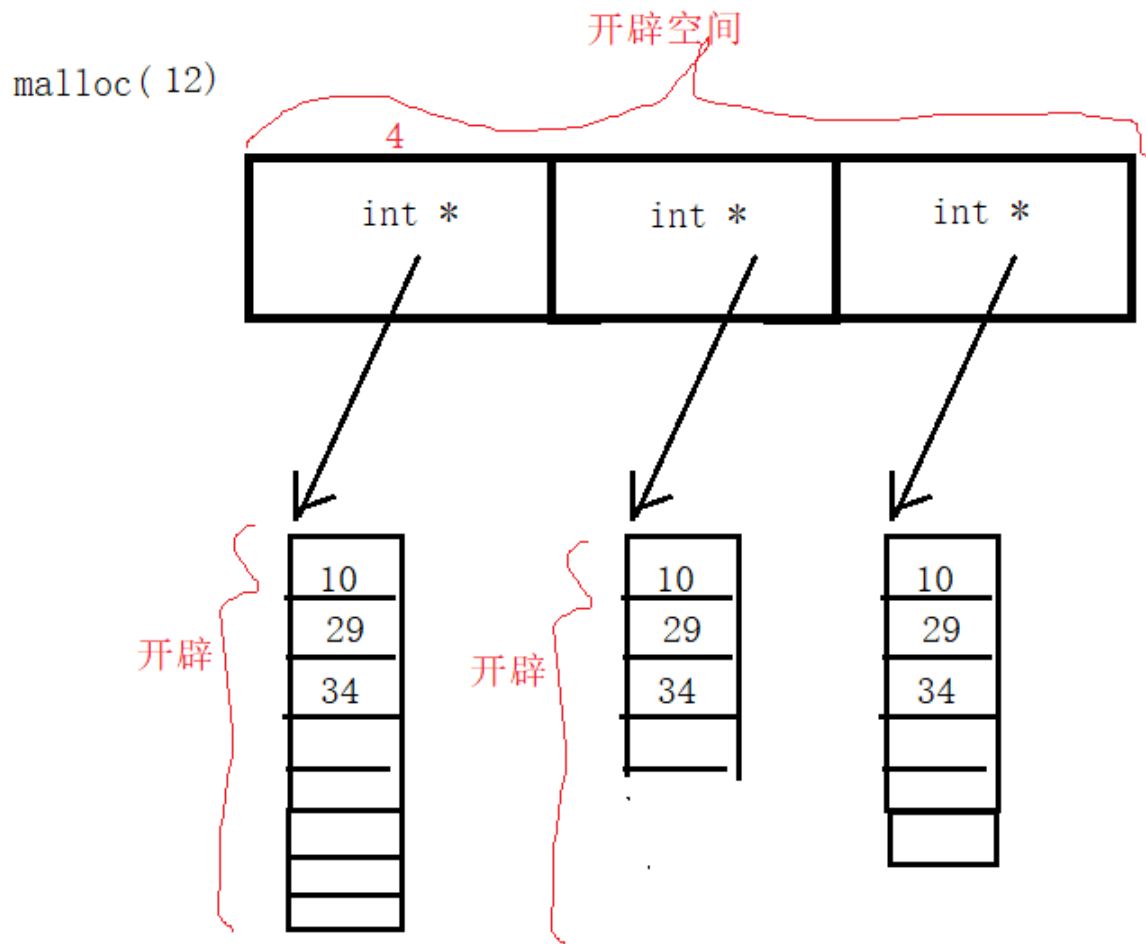


heap空间操作

二级指针对应的 heap 空间

- `int **p = int *p[3] ==> [int *, int *, int *] ==> [[1, 2, 3, 54, 5], int *, int *]`



```
1  int main(void)
2  {
3      // 给外层空间malloc申请内存
4      int** p = malloc(sizeof(int *) * 3);
5      if (p == NULL)
6      {
7          printf("malloc error!\n");
8          return -1;
9      }
10     //给 内层指针 申请 malloc 空间
11     for (size_t i = 0; i < 3; i++)
12     {
13         p[i] = malloc(sizeof(int) * 5);
14         if (p[i] == NULL)
15         {
16             printf("malloc p[i] error!\n");
17             return -1;
18         }
19     }
```

```

19     }
20     // 使用空间 -- 写
21     for (size_t i = 0; i < 3; i++)
22     {
23         for (size_t j = 0; j < 5; j++)
24         {
25             p[i][j] = i + j;    // 随意初始化值。
26         }
27     }
28     // 使用空间 -- 读
29     for (size_t i = 0; i < 3; i++)
30     {
31         for (size_t j = 0; j < 5; j++)
32         {
33             printf("%d ", (*(p + i) + j)); // p[i][j] == *(p+i)[j] == *((*
(p+i)+j)
34         }
35         printf("\n");
36     }
37
38     // free空间时，应该先释放 内层空间，再释放外层
39     for (size_t i = 0; i < 3; i++)
40     {
41         free(p[i]);
42         p[i] = NULL;
43     }
44
45     // 释放外层空间
46     free(p);
47     p = NULL;
48
49     system("pause");
50     return EXIT_SUCCESS;
51 }

```

以 char **p 为例

```

1 // 先申请外层指针。
2 char **p = malloc(sizeof(char *) * 5);
3 // 申请内层指针
4 for (int i = 0; i < 5; i++)
5 {
6     p[i] = malloc(sizeof(char) * 10); // 字符串长度 <= 10个字符。
7 }
8 // 写
9 for (int i = 0; i < 5; i++)
10 {
11     // p[i] = "hello"; 错误!
12     strcpy(p[i], "hello");
13 }
14 // 释放内层
15 for (int i = 0; i < 5; i++)
16 {
17     free(p[i]);
18     p[i] = NULL;

```

```

19 }
20 // 释放外层
21 free(p);
22 p = NULL;

```

内存操作函数

- 以下 4 个函数，专门用来操作 heap 内存。stack 由系统自动申请，自动释放。

memset

```

1 #include <string.h>
2 void *memset(void *s, int c, size_t n);
3     参1: 内存首地址。
4     参2: 置成什么。一般传 0
5     参3: 内存大小。单位：字节。
6     返回值：
7         成功：设置后的地址。
8         失败：NULL
9
10 memset(首地址, 0, 空间大小);

```

- 绝大多数，memset 用来将申请好的 heap 内存，置 0。—— 单位：字节。

```

1 int* p = (int *)malloc(sizeof(int) * 10);
2 if (p == NULL)
3 {
4     printf("malloc error");
5     return -1;
6 }
7 // 将申请好的内存，全部置 0
8 memset(p, 0, sizeof(int) * 10);
9
10 // 直接打印申请好的空间内容。
11 for (size_t i = 0; i < 10; i++)
12 {
13     printf("%d ", p[i]);
14 }
15
16 free(p);
17 p = NULL;

```

- memset 函数按“字节”设置。
 - 置0 ——> 每一个字节都为 0.
 - 【了解】置1 ——> 每一个字节都为 1. —— 4字节 == 0x01010101 == 16843009

memcpy

- 以字节为单元，内存拷贝。strcpy ——> 只能拷贝字符串。

```

1 void *memcpy(void *dest, const void *src, size_t n);

```

```

1 // 拷贝内存中的整型数据
2 int arr[10] = { 1, 2, 3, 4, 5 };
3 int arr2[10];
4
5 memcpy(arr2, arr, sizeof(int)*10);
6
7 for (size_t i = 0; i < 10; i++)
8 {
9     printf("%d ", arr2[i]);
10 }
11 printf("\n");
12
13 // 拷贝内存中的字符串
14 char str[] = "hello world";
15 char p[100];
16 //memcpy(p, str, strlen(str)+1); // 按内存拷贝
17 strcpy(p, str); // 字符串拷贝
18 strncpy(p, str, strlen(str) + 1);
19
20 printf("p = %s\n", p);

```

memmove

- 作用 完全等同于 memcpy。以字节为单元，内存拷贝。---- 安全的!
- 拷贝的 src 和 dest 之前如果有重叠，memcpy由于底层实现原因，有可能出错。推荐使用 memmove。

```

1 void *memmove(void *dest, const void *src, size_t n);

```

memcmp

- 以字节为单位，比较内存!
- 作用可以完全参照，strcmp。
- 规则: $s1 == s2 \rightarrow 0$, $s1 > s2 \rightarrow 1$ $s2 < s1 \rightarrow -1$

```

1 int memcmp(const void *s1, const void *s2, size_t n);

```

```

1 int arr1[] = { 14, 56, 22, 59, 21, 49, 25, 86, 89, 66, 71, 31, 98 };
2 int arr2[] = { 14, 56, 29, 59, 21 };
3
4 int ret = memcmp(arr1, arr2, 5 * sizeof(int));
5
6 printf("ret = %d\n", ret);

```

内存常见问题

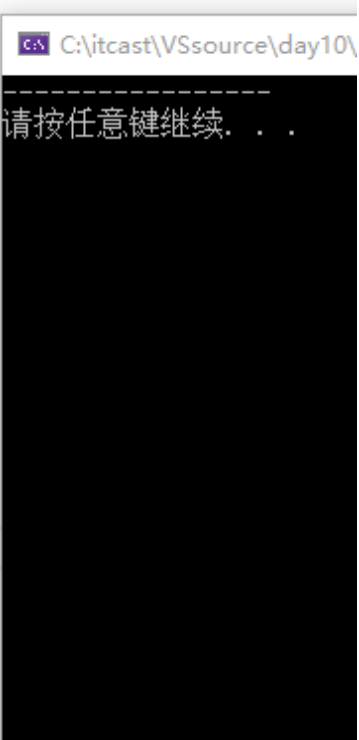
1. 申请 0 字节空间
 - C 语言中，允许申请 0 字节的内存空间。

```
int main(void)
{
    int* p = malloc(0);
    if (p == NULL)
    {
        printf("malloc error");
        return -1;
    }

    free(p);
    p = NULL;

    printf("-----\n");

    system("pause");
    return EXIT_SUCCESS;
}
```



- 0 字节的空间，不能拿来使用。
2. free 空指针
 - 空NULL指针，反复free，不会报错。
 - 非空的指针，反复free，**会报错!!!**——推荐 free 后的指针，一定置 NULL。
 3. 越界访问
 - 不允许!!! 导致程序崩溃。
 4. free ++ 后的地址
 - 不能正常释放。
 - 如果程序中，必须要使用 p++，定义临时变量，保存p值。以便free释放。

函数内申请空间使用。

A函数内申请空间，A函数使用

- 注意上述 4 点。

```
1 int *p = malloc(10);
2 free(p); p = NULL;
```

A函数内申请空间，B函数使用

- 例1

```
1 // A函数申请空间， B函数使用。
2 int* func1(void)
3 {
4     int* p = malloc(4);
5     *p = 456;
6     return p;    // 返回的 heap 堆空间的地址值，函数调用结束，地址 有效。
7     //return &p;    // 返回的 stack 栈空间的地址值，函数调用结束，地址 无效。
8 }
9 int main(void)
```

```

10 {
11     int* ret = NULL;
12     ret = func1();
13
14     printf("%d\n", *ret);
15     free(ret);
16     ret = NULL;
17
18     system("pause");
19     return EXIT_SUCCESS;
20 }

```

- 例2

```

1 // A函数申请空间， B函数使用。
2 int* func2(int *p)
3 {
4     p = malloc(4);
5     return p;
6 }
7
8 int main(void)
9 {
10     int* ret = NULL;
11     ret = func2(ret);
12
13     *ret = 345; // 写
14     printf("%d\n", *ret); // 读
15
16     free(ret);
17     ret = NULL;
18
19     system("pause");
20     return EXIT_SUCCESS;
21 }

```

- 例3

```

1 void func3(int** p) // int** 是一级指针的地址。
2 {
3     *p = malloc(4);
4 }
5
6 int main(void)
7 {
8     int* ret = NULL;
9
10    func3(&ret); // 传一级指针的地址。
11    // func3 函数，调用完成，ret指针，不再为 NULL，而是指向一块有效的 heap 空间地址。
12    *ret = 789;
13    printf("%d\n", *ret);
14
15    free(ret);
16    ret = NULL;

```

```
17
18     system("pause");
19     return EXIT_SUCCESS;
20 }
```

结构体

结构体定义语法

- 复合类型：用户自定义类型：int []、int *、char **、struct student

```
1 // 定义结构体类型
2 struct student {
3     int age;           // 成员变量 —— 属性。不能被赋初值。
4     int num;
5     char name[10];
6 };
7 // 用结构体类型，定义变量。
8 struct student a;     // 定义了一个结构体类型的变量 a。
```

- 以上定义了一个结构体**类型**。名字叫 struct student。
- struct student 的地位 等同于 int、char、short、char *、int[]、long long

- 通常 结构体类型定义在 全局位置。 或者 放到 xxx.h 头文件。
- 头文件：

```
1 // 头文件守卫
2 #ifndef _XXX_H_
3 #define _XXX_H_
4 4种: #include、宏定义、函数声明、类型定义（结构体类型）。
5 #endif
```

普通结构体变量

定义语法

```
1 struct student stu1, stu2, stu3; // 一次定义3个变量。没赋初值。
2 struct student stu = {18, 1, "Andy"};
```

访问成员方法

- 使用 "." 访问成员。

```

1 // 定义一个结构体变量，赋初值。
2 struct student stu = {18, 1, "Andy"};
3
4 printf("first: age = %d, name = %s, num = %d\n",
5       stu.age, stu.name, stu.num);
6
7 stu.age = 118;
8 stu.num = 119;
9 //stu.name = "cuihua"; // name为地址常量，不能被赋值。
10 strcpy(stu.name, "cuihua");
11
12 printf("last: age = %d, name = %s, num = %d\n",
13       stu.age, stu.name, stu.num);

```

- 普通变量使用“->”访问成员 ---- 不常用!

```

1 (&stu)->age = 118;
2 (&stu)->num = 119;

```

结构体指针变量

定义语法

```

1 struct student *p1, *p2, *p3; // 一次定义3个指针变量。 野指针!!!

```

访问成员方法

- 使用“->”访问成员。
- 避免野指针、空指针:

```

1. 1 struct student stu, *p1; //一次定义两个结构体变量，一个普通变量stu，另一指针
   2 变量p1
   2 p1 = &stu; // 给指针初始化。

```

```

2. 1 struct student *p1;
   2 p1 = (struct student *)malloc(sizeof(struct student));

```

- 指针使用“.”访问成员。 ---- 不常用!

```

1 (*p1).age = 18;
2 strcpy((*p1).name, "cuihua");
3 (*p1).num = 119;

```

非常规定义语法（了解）


```

1 struct student {
2     int age;
3     int num;
4     char name[10];
5 }s1, *s2;    // 定义结构体类型的同时，定义1个结构体变量 s1，一个指针变量s2
6
7 struct {      // 匿名结构体
8     int age;
9     int num;
10    char name[10];
11 }s3, *s4; // 定义匿名结构体类型的同时，定义1个结构体变量 s3，一个指针变量s4。
12
13          // 无法再定义其他变量。

```

结构体数组

```

1 struct student {
2     int age;
3     int num;
4     char name[10];
5 };
6 struct student stu[5] = {{18, 1, "Andy"}, {19, 2, "Lucy"}, {118, 3, "李
  四"}};
7 int n = sizeof(stu) / sizeof(stu[0]);
8 for (int i = 0; i < n; i++)
9 {
10     printf("age=%d,num=%d,name=%s\n", stu[i].age, stu[i].num, stu[i].name);
11 }

```

- struct student *stu; // 要求 指针指向能存储3个student 元素的空间，并给3个元素赋初值。访问。

```

1 struct student *stu;    // 野指针
2
3 // 得到的heap堆空间，当成数组使用。
4 stu = malloc(sizeof(struct student)*3); // 等价于 struct student stu[3];
5
6 // 给数组的第1个元素赋值。
7 stu[0].age = 11;
8 stu[0].num = 111;
9 strcpy(stu[0].name, "aaa");
10
11 // 给数组的第2个元素赋值。
12 stu[1].age = 22;
13 stu[1].num = 222;
14 strcpy(stu[1].name, "bbb");
15
16 // 给数组的第3个元素赋值。
17 stu[2].age = 33;
18 stu[2].num = 333;
19 strcpy(stu[2].name, "ccc");
20
21 //int n = sizeof(stu) / sizeof(stu[0]); // 不能求元素个数。
22
23 for (int i = 0; i < 3; i++)

```

```
24 {
25     printf("age=%d,num=%d,name=%s\n", stu[i].age, stu[i].num, stu[i].name);
26 }
27
28 free(stu);
29 stu = NULL;
```

结构体嵌套

```
1 struct person {
2     int age;
3     char name[10];
4 }; // 类型
5
6 struct student {
7     struct person man; // person 类型的变量，作为 student 类型成员。
8     int id;
9     char addr[100];
10 };
11
12 int main(void)
13 {
14     struct student stu = {{18, "zhaoliu"}, 1, "北京朝阳区"};
15
16     printf("age = %d\n", stu.man.age);
17     printf("name = %s\n", stu.man.name);
18     printf("addr = %s\n", stu.addr);
19
20     // 修改
21     stu.man.age = 119;
22     strcpy(stu.man.name, "张三丰");
23     strcpy(stu.addr, "武当山");
24
25     printf("\nage = %d\n", stu.man.age);
26     printf("name = %s\n", stu.man.name);
27     printf("addr = %s\n", stu.addr);
28
29     system("pause");
30     return EXIT_SUCCESS;
31 }
```

做函数参数、返回值

结构体变量赋值

- 主要应用于，函数调用期间，实参给形参赋值。
- 要求：
 - 结构体变量赋值时，必须类型相同、成员个数一致，顺序一致。

做参数、返回值

- 传值：
 - 结构体变量做函数参数，将结构体变量的值(实参)，拷贝一份给形参。
 - 形参、实参 共 2 份结构体。
- 传址：
 - 结构体指针变量做函数参数，将结构体的地址值做实参，拷贝一份给形参。
 - 形参、实参 共 1 份结构体。
- 结论：
 - 结构体做函数参数、返回值时，通常采用“传址”方式，节省空间。

```

1 void func08(struct student **m)
2 {
3     *m = malloc(sizeof(struct student));
4     if (NULL == *m)
5     {
6         printf("malloc error\n");
7         return -1;
8     }
9     //p->age = 100;
10    //p->num = 1;
11    //strcpy(p->name, "zyx");
12    (*m)->age = 100;
13    (*m)->num = 1;
14    strcpy((*m)->name, "zyx");
15 }
16
17 int main(void)
18 {
19     struct student* p = NULL; // 空指针
20
21     func08(&p);
22
23     //p->age = 100;
24     //p->num = 1;
25     //strcpy(p->name, "zyx");
26
27     printf("age=%d, name=%s, num=%d\n", p->age, p->name, p->num);
28
29     free(p);
30     p = NULL;
31
32     system("pause");
33     return EXIT_SUCCESS;
34 }

```

含有指针成员的结构体

- 申请内存：先申请外层空间，再申请内层空间。
- 释放内存：先释放内存空间，在释放外层空间。

```

1 struct student {
2     int age;

```

```

3     int num;
4     char *name;    // 野指针。
5 };
6
7 int main(void)
8 {
9     struct student* p;    //野指针。
10
11    // 给 p 初始化堆空间
12    p = malloc(sizeof(struct student));
13    if (NULL == p)
14    {
15        printf("malloc p error\n");
16        return -1;
17    }
18    // 给成员变量 name 开辟堆空间
19    p->name = malloc(sizeof(char) * 100);
20    if (NULL == p->name)
21    {
22        printf("malloc p->name error\n");
23        return -1;
24    }
25    // 写数据到结构体中
26    p->age = 100;
27    p->num = 10;
28    strcpy(p->name, "张三丰");
29
30    printf("age=%d, name=%s, num=%d\n", p->age, p->name, p->num);
31
32    // 先释放内层空间
33    free(p->name);
34    p->name = NULL;
35
36    free(p);
37    p = NULL;
38
39    return 0;
40 }

```

typedef 关键字

- 给现有的数据类型起别名。【注意】：不能定义新数据类型。

```

1 typedef unsigned int size_t; // 给 unsigned int 起别名叫 size_t
2
3 int a; a 是变量名。
4 typedef int a; a 变成了 类型名。 a b; 定义一个整型变量 b。（可读性差）

```

- 通常使用 typedef 定义过的类型，添加一个“_t”结尾。
- 定义语法：
 - typedef 旧类型名 新类型名_t;

```

1 typedef struct student {
2     int age;
3     int num;
4     char *name;    // 野指针。
5 } stu_t;           // 新类型名: stu_t;
6 // 定义变量
7 struct student stu1; // 依然可以正常使用
8 stu_t stu2;         // 定义一个 struct student 类型的变量。

```

- 使用 typedef 的好处:

1. 简化类型名。
2. 便于代码的修改和维护。

```

1 typedef long long int32_t;    // int 《----》 long long
2
3 struct student {
4     int age;
5     int32_t num;
6     char *name;    // 野指针。
7     int32_t num1;
8     int32_t num2;
9     int32_t num3;
10    int32_t num4;
11 } stu_t;

```

共用体 (联合体)

```

1 union test {
2     char ch;
3     short sh;
4     int var;
5 };    // 创建一个联合体类型。

```

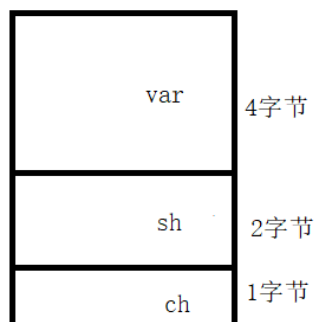
- 特性:

- 内部所有成员变量的地址一致。等同于整个联合体的地址。
- 联合体的大小, 是内部成员变量中, 最大的那个成员的大小。(对齐)
- 修改其中一个成员的值, 其他成员的值也跟着变化。

```

struct test {
    char ch;
    short sh;
    int var;
};

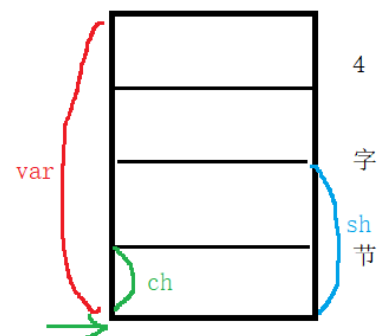
```



```

union test {
    char ch;
    short sh;
    int var;
};

```



```
1  typedef union test {
2      char ch;
3      short sh;
4      int var;
5  } test_t;
6
7  int main(void)
8  {
9      test_t obj;
10
11     obj.var = 0x87654321;
12
13     printf("&obj    = %p\n", &obj);
14     printf("&obj.ch = %p\n", &obj.ch);
15     printf("&obj.sh = %p\n", &obj.sh);
16     printf("&obj.var= %p\n", &obj.var);
17
18     printf("sizeof(test_t) = %u\n", sizeof(test_t));
19
20     printf("var = %#x\n", obj.var);
21     printf("sh = %#x\n", obj.sh);
22     printf("ch = %#x\n", obj.ch);
23
24     obj.ch = 0xAA;
25
26     printf("var = %#x\n", obj.var);
27     printf("sh = %#x\n", obj.sh);
28     printf("ch = %#x\n", obj.ch);
29
30     system("pause");
31     return EXIT_SUCCESS;
32 }
```